

DBMS Lecture 7: Integrity Constraints

Domain, Entity, Referential & Key Constraints

Topper's Revision Notes | Visual Diagrams, SQL Implementations & GATE Shortcuts

1. Introduction

When building a database, we organize data across multiple relational tables. However, a system cannot allow users to insert random, non-sensical, or invalid values.

Consider a basic Student table:

Roll_No	Name	Age
101	Rahul	20

Allowing an entry such as (101, "Rahul", -5) would be incorrect because a student's age logically cannot be negative. Therefore, a database management system enforces specific rules to control data insertion, modification, and deletion.

Definition (★)

Integrity Constraints are predefined rules enforced on a database to maintain the accuracy, consistency, validity, and reliability of stored data.

In simple words: Integrity constraints act as structural guardrails, preventing invalid or contradictory data from entering or persisting in the database.

2. Why Do We Need Integrity Constraints?

Without constraints, users could insert arbitrary data into tables, leading to critical data corruption:

- Age = -5 Or Age = 500
- Roll_No = NULL
- Duplicate Roll_No for different students
- Course_ID = 9999 (pointing to a non-existent course)

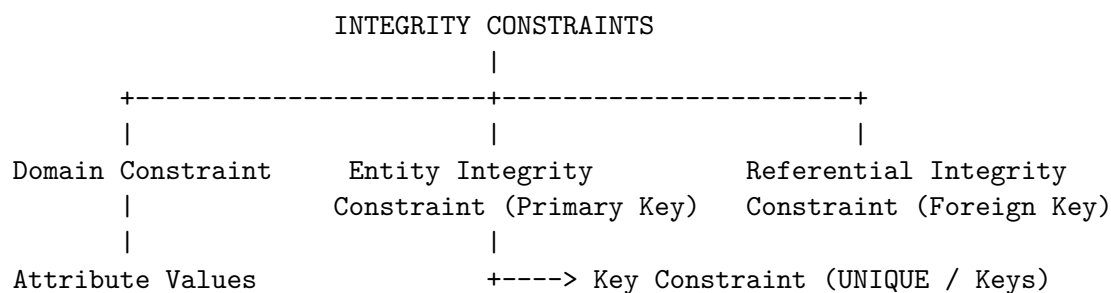
Core Pillars of Data Quality

Enforcing constraints guarantees the following core properties:

- **Accuracy:** Data reflects real-world valid information.
- **Consistency:** Related figures across tables remain synchronized.
- **Validity:** Values conform strictly to business domain constraints.
- **Reliability:** Client applications can query the database with total trust.

3. Types of Integrity Constraints (★★★ High Frequency)

Integrity constraints are structured into four principal categories:



The key types studied in standard database courses are:

1. **Domain Constraint**
2. **Entity Integrity Constraint**
3. **Referential Integrity Constraint**
4. **Key Constraint**

4. Domain Constraint

Definition

A **Domain Constraint** restricts the set of permissible values that an attribute can hold. It defines the allowed data type, format, and value range for a given column.

4.1 Attribute Types

Given a table `Student(Roll_No, Name, Age)`:

- `Roll_No → INTEGER`

- Name → VARCHAR / STRING
- Age → INTEGER

Inserting Age = "ABC" fails domain verification, whereas Age = 20 succeeds.

4.2 Age & CHECK Constraints

To restrict numeric values to a valid physical range (e.g., Age > 0):

```
CREATE TABLE Student (
    Roll_No INT,
    Name VARCHAR(50),
    Age INT CHECK (Age > 0)
);
```

- Age = 20 → Valid ✓
- Age = -5 → Rejected by DBMS ✗

4.3 Mobile Number Domain Rule

If a mobile phone number must contain exactly 10 numeric digits:

Length(Mobile_Number) = 10

Entries such as 9876543210 pass domain validation, while truncated strings like 98765432 are rejected.

5. Entity Integrity Constraint

Definition (★)

The **Entity Integrity Constraint** mandates that the ****Primary Key**** of a relation must contain unique values and can ****never be NULL****.

Every tuple in a relational table must be uniquely identifiable. This identity is enforced via a Primary Key.

5.1 Properties of a Primary Key

1. **Uniqueness:** No two distinct rows can share the exact same primary key value.
2. **NOT NULL:** Primary key attributes cannot contain missing or undefined (NULL) entries.

Roll_No Validation:

```
101 --> Valid
102 --> Valid
101 --> Invalid (Duplicate)
NULL --> Invalid (Violates NOT NULL)
```

```
CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

Why Entity Integrity Matters

If multiple rows shared a single primary key or had NULL identifiers, identifying specific records for updates, deletions, or joins would become impossible.

6. Referential Integrity Constraint (★★★ Critical)

Definition

Referential Integrity ensures that a Foreign Key value in a child table must reference a valid, existing key value in the parent table (or be NULL, if permitted).

6.1 Parent Table vs. Child Table

Consider a scenario where students enroll in university courses:

Parent Table: Course			Child Table: Student		
Course_ID	Course_Name		Student_ID	Name	Course_ID
1	BCA		101	Rahul	1
2	B.Tech		102	Ravi	2
3	MCA				

Relationship: Student.Course_ID → Course.Course_ID

- Course is the **Parent Table** (Referenced Relation).
- Student is the **Child Table** (Referencing Relation).
- Student.Course_ID is the **Foreign Key**.

6.2 Foreign Key SQL Implementation

```
CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50)
);
```

```
CREATE TABLE Student (
```

```

Student_ID INT PRIMARY KEY,
Name VARCHAR(50),
Course_ID INT,
FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

```

If an insert attempts to add a student with `Course_ID = 10`, the DBMS checks the parent `Course` table. Since `Course_ID = 10` does not exist, the operation is blocked.

7. Key Constraint & UNIQUE Keyword

Definition

A **Key Constraint** guarantees that candidate key attributes contain unique values across all tuples in a relation.

Attributes such as `Email`, `Phone_Number`, and `Aadhaar_No` must remain unique per user without necessarily being chosen as the primary key.

```

CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);

```

Key Constraint vs. Primary Key

Dimension	Key Constraint (UNIQUE)	Primary Key
Purpose	Enforces uniqueness on candidate attributes.	Serves as the primary identifier for rows.
NULL Values	May permit NULL values (depending on RDBMS).	Strictly CANNOT be NULL.
Quantity	A table can have **multiple** UNIQUE columns.	A table has **at most one** primary key.

Table 1: Key Constraint vs. Primary Key

8. Complete System Example

Consider a university schema with `Department` and `Student` relations:

DEPARTMENT (Parent)			STUDENT (Child)			
Dept_ID	Dept_Name		Roll_No	Name	Age	Dept_ID
10	Computer Science		101	Rahul	20	10
20	Commerce		102	Ravi	21	20

8.1 SQL Implementation

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50) UNIQUE  
);  
  
CREATE TABLE Student (  
    Roll_No INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age > 0),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

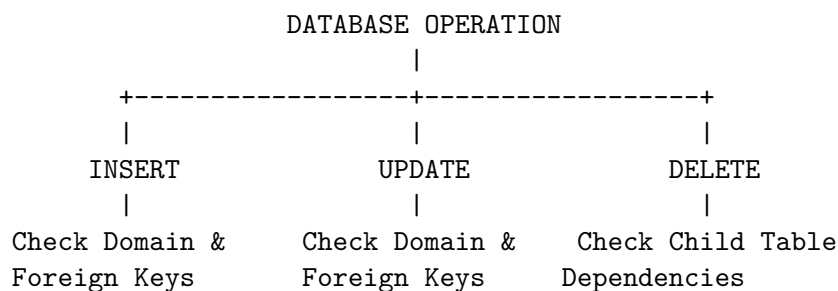
8.2 Constraint Mapping Matrix

Constraint Category	Attribute Target	SQL Mechanism / Rule
Domain Constraint	Age INT	CHECK (Age > 0) restricts value range.
Entity Integrity	Roll_No INT	PRIMARY KEY (Unique + NOT NULL).
Key Constraint	Dept_Name	UNIQUE prevents duplicate department names.
Referential Integrity	Student.Dept_ID	FOREIGN KEY REFERENCES Department(Dept_ID).
Nullity Constraint	Name	NOT NULL prevents missing names.

Table 2: Comprehensive Constraint Mapping

9. Operations & Referential Integrity

Database operations (INSERT, UPDATE, DELETE) trigger automatic constraint validation checks:



- **INSERT Execution:** Adding (103, "Sara", -2, 10) triggers a Domain violation (Age > 0).
- **UPDATE Execution:** Modifying Dept_ID = 50 for a student fails if Dept_ID = 50 does not exist in Department.

- **DELETE Execution:** Deleting Dept_ID = 10 from Department is blocked if child rows in Student reference department 10 (unless cascade rules are defined).

10. Quick Comparison of Constraints

Constraint Type	Primary Purpose	Core Target	SQL Syntax
Domain	Restrict data types & value ranges	Attributes	Data Types, CHECK
Entity Integrity	Ensure unique row identification	Primary Key	PRIMARY KEY
Referential Integrity	Maintain parent-child relationships	Foreign Key	FOREIGN KEY ... REFERENCES
Key Constraint	Guarantee value uniqueness	Candidate Keys	UNIQUE

Table 3: Quick Reference: Summary of DBMS Constraints

11. Top 10 Interview & GATE Questions

Q1. What are integrity constraints?

Answer: Predefined rules enforced by a DBMS to maintain accuracy, consistency, validity, and reliability across stored records.

Q2. What is a domain constraint?

Answer: A constraint restricting an attribute to a specific set or range of valid data values (e.g., Age > 0).

Q3. What does Entity Integrity mandate?

Answer: It mandates that every relation's Primary Key must be unique and contain no NULL entries.

Q4. Define Referential Integrity.

Answer: A rule stating that foreign key values in a child table must reference valid existing primary key values in the referenced parent table.

Q5. Which key corresponds to Entity Integrity?

Answer: Primary Key.

Q6. Which key corresponds to Referential Integrity?

Answer: Foreign Key.

Q7. Can a primary key accept NULL entries?

Answer: No. Entity integrity strictly forbids NULL values in primary keys.

Q8. Differentiate between Data Types and Constraints.

Answer: A data type defines the **kind** of data allowed (e.g., INT), whereas a constraint defines **rules** governing allowed values (e.g., CHECK (Age > 0)).

Q9. What is the role of the UNIQUE constraint?

Answer: It ensures that no duplicate entries exist for specified non-primary key columns.

Q10. What occurs if a foreign key references a non-existent parent value?

Answer: The DBMS blocks the operation and throws a Referential Integrity violation error.

12. Rapid Revision & Exam Flashcards

The D-E-R-K Memory Formula

Remember the acronym **D-E-R-K**:

- **D** → **Domain Constraint**: What values are allowed? (Data Types, CHECK)
- **E** → **Entity Integrity**: Primary Key ⇒ **Unique + NOT NULL**
- **R** → **Referential Integrity**: Foreign Key ⇒ Must reference valid Parent Key
- **K** → **Key Constraint**: Uniqueness ⇒ **UNIQUE Keyword**

DBMS Study Roadmap Progress

- Lecture 1: Database → DBMS → RDBMS
- Lecture 2: File System vs DBMS
- Lecture 3: 2-Tier & 3-Tier Architecture
- Lecture 4: Schema in Database
- Lecture 5: Three-Schema Architecture & Data Abstraction
- Lecture 6: Data Independence (Logical & Physical)
- **Lecture 7: Integrity Constraints (COMPLETED ✓)**
- *Next Lecture: Keys in DBMS — Super Key, Candidate Key, Primary Key, Alternate Key & Foreign Key*